

Debkanta Study Zone – Notes by Debkanta Maity

OOPs – Unit 2: Object Oriented Programming Systems

Topics covered:

- Classes & Objects
- Constructors & Destructors
- Inheritance
- Polymorphism
- Encapsulation
- Dynamic Binding

Example code (from cover page):

```
class Animal {
    public void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    @Override
    public void eat() {
        System.out.println("Dog is eating...");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.eat();
    }
}
```

Unit - 2

SYLLABUS: Features of object-oriented programming. Encapsulation, Object identity, polymorphism – but not inheritance

Features of Object-Oriented Programming

The core features, often referred to as the Four Pillars of Object-Oriented Programming (OOP), are Encapsulation, Abstraction, Inheritance, and Polymorphism.

Key Features (The Four Pillars)

1. Encapsulation:

- This is the mechanism of bundling the data (attributes) and the methods (functions) that operate on that data into a single unit, which is the class.
- It also involves data hiding, restricting direct access to some of an object's components (typically making data fields private) and exposing them only through well defined public methods (like getters and setters).

Benefit: Protects the integrity of the data from unintended external modification.

2. Abstraction:

- This involves showing only the essential information to the user and hiding the complex implementation details.
- Think of it like driving a car: you interact with the steering wheel and pedals (the essential interface) without needing to know the complex internal workings of the engine and transmission.

Benefit: Promotes code reusability and avoids defining the same properties and methods repeatedly.

3. Inheritance:

- This allows a new class (subclass/child class) to adopt the properties and behaviours (methods and attributes) of an existing class (superclass/parent class).
- It creates a hierarchical "is-a" relationship (e.g., a Dog is a type of Animal).

Benefit: Promotes code reusability and avoids defining the same properties and methods repeatedly.

4. Polymorphism:

- Meaning "many forms," polymorphism allows an entity (like a function or an operator) to be treated differently or take on multiple forms depending on the context.
- A common example is having the same method name, say draw(), behave differently in different classes (e.g., a Circle's draw() method draws a circle, while a Square's draw() method draws a square).
- It can be achieved through Method Overloading (compile-time) and Method Overriding (runtime).

Benefit: Increases flexibility and allows for a single interface to handle different underlying data types or classes.

Supporting Concepts:

Classes: These are blueprints or templates for creating objects. They define the structure (data/attributes) and behaviour (methods) that all objects of that class will possess.

Objects: These are instances of a class. They are real-world entities that possess the attributes and behaviours defined by their class.

Short Note on Encapsulation

Meaning:

Encapsulation is a fundamental concept of Object-Oriented Programming (OOP) where data (attributes) and the methods (functions) that operate on that data are bundled together into a single unit (class). It ensures that the internal representation of an object is hidden from the outside world.

Key Idea:

- Encapsulation = Data + Methods wrapped together + Controlled Access
- It is implemented using access modifiers (private, public, protected).

How Encapsulation Works:

- Make data members private so they cannot be accessed directly from outside the class.
- Provide public getter and setter methods to access or modify the data safely.
- Users interact with the object only through these methods, not directly with the internal data.

Example (Java):

```
class Employee {  
    private double salary; // private data -> hidden  
    public void setSalary(double s) { // setter method  
        salary = s;  
    }  
    public double getSalary() { // getter method  
        return salary;  
    }  
}
```

Objectives / Purpose of Encapsulation:

- **Data Hiding:** Prevents accidental or unauthorized access to data.
- **Security:** Sensitive data like passwords, account numbers, etc., are protected.
- **Controlled Access:** Allows validation before changing data (e.g., age cannot be negative).
- **Modularity:** Each class manages its own data.

- **Maintainability:** Internal code can be changed without affecting other parts of a program.
- **Flexibility:** Methods can be modified while keeping data hidden.

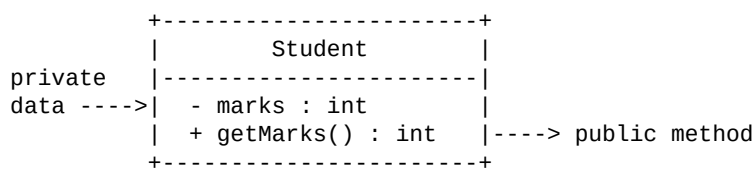
Advantages:

- Protects data from misuse.
- Helps in achieving abstraction.
- Reduces system complexity.
- Makes the program easier to debug and manage.
- Enhances code reusability.

Real-Life Example:

A bank ATM: You can check balance or withdraw money, but you cannot directly access or modify bank databases. Internal details are hidden – interaction happens only through provided options (methods).

Diagram – Encapsulation:



Short Note on Abstraction

Meaning:

Abstraction is an important principle of Object-Oriented Programming (OOP) that focuses on showing only essential features of an object while hiding unnecessary details. It simplifies complex systems by exposing only what is needed.

Key Idea:

- Abstraction = Hiding internal implementation + Showing only relevant functionality
- It deals with what an object does, not how it does it.

Implementation in Java:

Abstraction is mainly achieved using:

- Abstract classes
- Interfaces

Both allow you to define functions without complete implementation, forcing subclasses to provide details.

Example (Abstract Class):

```

abstract class Vehicle {
    abstract void start(); // only method definition
}
class Car extends Vehicle {
    void start() {
        System.out.println("Car starts with a key.");
    }
}

```

Example (Interface):

```

interface Animal {
    void sound();
}

```

The interface provides only essential behaviour. Actual implementation is hidden inside classes like Dog, Cat etc.

Purpose of Abstraction:

- To reduce complexity.
- To hide unnecessary implementation details from the user.
- To provide a clear and simple view of objects.

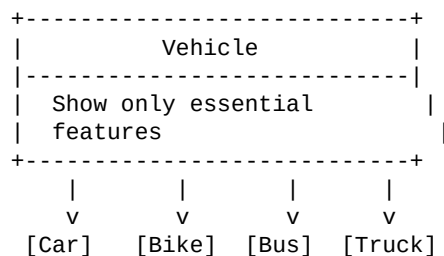
Advantages of Abstraction:

- Simplifies complex systems by breaking them into smaller, manageable parts.
- Enhances security, because internal logic is hidden.
- Improves code reusability.
- Helps in achieving loose coupling.
- Easier maintenance since implementation can change without affecting users.

Real-Life Examples:

- **Car Driving:** You use steering, brake, accelerator – but don't know the internal engine mechanics.
- **ATM Machine:** You press buttons (withdraw, check balance), but the internal banking logic is hidden.

Diagram – Abstraction:



Short Note on Inheritance

Meaning:

Inheritance is an important OOP concept where one class acquires the properties and behaviours of another class. The class that is inherited from is called the parent/super/base class, and the class that inherits is called the child/sub/derived class.

Key Idea:

- It supports code reusability.
- Creates a hierarchical relationship between classes.
- Allows a class to extend or modify behaviours of another class.

Syntax in Java:

```
class Parent {
    void display() {
        System.out.println("Parent class");
    }
}
class Child extends Parent { // inheritance using 'extends'
}
```

Here, child automatically gets the functionality of Parent.

Types of Inheritance in Java:

1. Single Inheritance

A class inherits from one parent class. A -> B

```
class A { }
class B extends A { }
```

2. Multilevel Inheritance

A class inherits from a derived class, forming a chain. A -> B -> C

```
class A { }
class B extends A { }
class C extends B { }
```

3. Hierarchical Inheritance

Multiple child classes inherit from a single parent.



```
class A { }
class B extends A { }
class C extends A { }
```

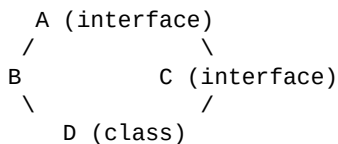
4. Multiple Inheritance (Through Interfaces Only)

A class can inherit behaviour from multiple interfaces, but not multiple classes.

```
interface X { }
interface Y { }
class A implements X, Y
```

5. Hybrid Inheritance (Using Interfaces)

Combination of two or more types (like hierarchical + multiple). Java supports hybrid inheritance only through interface.



Purpose / Objectives of Inheritance:

- **Code Reusability:** No need to rewrite common code in multiple classes.
- **Extensibility:** Existing classes can be extended with extra features.
- **Maintenance:** Changes in parent class automatically reflect in child classes.
- **Polymorphism Support:** Inheritance helps achieve runtime polymorphism (method overriding).

Example – Method Overriding:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Advantages of Inheritance:

- Reduces code duplication.
- Simplifies complex code.
- Promotes reusability.
- Supports abstraction and polymorphism.
- Enhances readability.

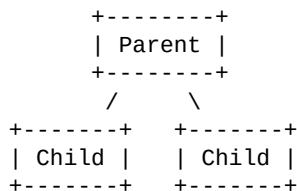
Disadvantages:

- Increases dependency between classes.
- Incorrect use may lead to tight coupling.
- Java does not support multiple inheritance with classes (to avoid ambiguity).

Real-Life Example:

Vehicle -> Car -> Sports Car. A car is a Vehicle, and a Sports Car is a more specific version of Car.

Diagram – Inheritance:



Short Note on Polymorphism

Meaning:

Polymorphism is an OOP concept that allows a single entity (method, object, or operator) to behave in different forms. The word polymorphism means "many forms". It enables the same method name to perform different tasks depending on the object that calls it.

Purpose:

- To achieve flexibility and dynamic behaviour in programs.
- To allow the same interface to be used for different data types.
- To support code reusability and clean structure.
- To simplify code maintenance.

Advantages of Polymorphism:

- Enhances readability and maintainability.
- Allows objects to be treated as instances of their parent class.
- Helps implement method overriding and enable dynamic binding.
- Reduces code duplication.

Types of Polymorphism in Java:

1. Compile-time Polymorphism (Static Polymorphism / Early Binding)

Achieved using Method Overloading.

Method Overloading: Multiple methods in the same class having:

- same method name
- different parameters (type, number or order)

Example:

```

class MathOp {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}

```

This is resolved at compile time.

2. Runtime Polymorphism (Dynamic Polymorphism / Late Binding)

Achieved using Method Overriding.

Method Overriding: Child class provides its own implementation of a method already present in the parent class.

Example:

```
class Animal {  
    void sound() { System.out.println("Animal makes sound"); }  
}  
class Dog extends Animal {  
    void sound() { System.out.println("Dog barks"); }  
}
```

This is resolved at runtime using dynamic method dispatch.

Dynamic Method Dispatch

It is the mechanism by which an overridden method is called through a parent class reference pointing to a child class object.

Example:

```
Animal a = new Dog();  
a.sound(); // calls Dog's sound()
```

The method executed depends on object type, not reference type.

Real-Life Examples:

- A remote control can operate TV, AC, or music system (same interface, different behaviour).
- A shape class may have a draw() method, implemented differently by Circle, Square, Triangle.

Difference between Overloading and Overriding

Feature	Overloading	Overriding
Type	Compile-time	Runtime
Class	Same class	Different classes
Parameters	Must differ	Must be same
Inheritance	Not required	Required
Purpose	Performance/Convenience	Runtime flexibility

Difference between Object Identity and Object Equality

Feature	Object Identity	Object Equality
Meaning	Same object in memory.	Same content/state
Comparison method	==	.equals()
Checks	Memory address	Field values
Default Behaviour	Always available	Needs method override for custom classes.
Example outcome	s1 == s2 may be false	s1.equals(s2) may be true.
Focus	"Are they the same object?"	"Do they look the same?"

Explain various access modifiers (public, protected, private, default) in a class definition.

=> Access modifiers are keywords in Object-Oriented Programming (specifically in languages like Java) that set the accessibility or scope of a class, field (variable), method, or constructor. They are vital for enforcing encapsulation, one of the core principles of OOP.

1. private (Most Restrictive)

- **Visibility:** Members are visible only inside the class where they are declared. They are completely hidden from the outside world.
- **Data Hiding:** This is the primary mechanism for achieving data encapsulation. Class fields are typically declared private, and controlled access is provided through public getter/setter methods.
- **Example:** A private int salary field in an Employee class.

2. default (Package-Private)

- **Visibility:** If you don't use any explicit access modifier, the member is given the default access level. It is accessible to all classes within the same package.
- **Top-level Classes:** Note that top-level classes (classes not nested within another class) can only be declared as public or default.
- **Example:** If a class Helper is declared without an access modifier, it can only be instantiated and used by other classes in the same package.

3. protected

- **Visibility:** It provides access within the same package (like default) plus one crucial extension: it allows access to subclasses of the class, even if those subclasses are located in a different package.
- **Inheritance:** The modifier is specifically designed to facilitate inheritance, allowing child classes access to parent class members needed for specialized behaviour.
- **Example:** A protected void calculateBonus() method in a base Employee class that subclasses need to override.

4. public (Least Restrictive)

- **Visibility:** Members are accessible from everywhere – the same class, the same package, subclasses, and completely unrelated classes in different packages.
- **Interface:** This modifier defines the public-facing Application Programming Interface (API) of your class. Methods and constructors necessary for external use are typically declared public.
- **Example:** A public void startEngine() method in a Car class.

Difference between protected and friendly access specifier

Parameter	protected Access Modifier	default (Friendly/Package-Private) Access Modifier
Keyword	Uses the keyword protected.	No keyword is used; it is applied when no modifier is explicitly specified.
Scope within Package	Accessible to all classes within the same package.	Accessible to all classes within the same package.
Scope outside Package	Accessible to subclasses (child classes) of the class, even if the subclass is in a different package.	Not accessible outside the package, even if the class attempting access is a subclass.
Applicability to class	Cannot be applied to top-level classes.	Can be applied to top-level classes.
Principle	Facilitates Inheritance. Allows child classes to access or override essential parent members.	Enforces Package Encapsulation. Members are private to the component (package) they belong to.

Association

Association is a fundamental concept in Object-Oriented Programming (OOP) that describes a relationship between two or more independent classes. It represents a structural link where objects of one class can use or interact with objects of another class.

In essence, association is a generic term for any relationship between classes. It is often described as a "has-a" relationship.

1. Defining Association

- **Relationship:** It's a static, structural relationship between classes.
- **Implementation:** In Java, association is typically implemented using instance variables (fields) that reference objects of other classes.
- **Direction:** Association can be unidirectional (one-way street, where class A knows about Class B, but B doesn't know about A) or bidirectional (two-way street, where both classes know about each other).
- **Example:** A Teacher has an association with a Student. The Teacher class might have a list of Student objects.

2. Properties of Association

- **Name:** Describes the purpose of the relationship (e.g., "teaches," "works for.").
- **Multiplicity:** Defines the number of objects participating in the relationship (e.g., one-to-one, one-to-many, many-to-many).
- **Role:** Specifies the role of each class in the relationship (e.g., in a Company-Employee association, Company plays the role of "employer" and Employee plays the role of "worker").

Types of Association:

Association is a broad category that includes two specific, stronger types of relationships: Aggregation and Composition. These are differentiated by the strength and lifecycle dependency of the relationship.

A. Aggregation (Weak "Has-A")

Aggregation represents a weak form of association where one object (the "whole") contains or consists of other objects (the "parts").

- **Dependency:** It is a weak or non-dependent relationship.
- **Lifecycle:** The "part" object can exist independently of the "whole" object. If the "whole" is destroyed, the "part" still survives.
- **UML:** Represented by a hollow diamond at the "whole" side of the association line.
- **Example:** A Department has a relationship with a Professor. If the Department is closed (destroyed), the Professor will exist (is not destroyed) along with the department.

B. Composition (Strong "Has-A")

Composition represents a strong form of association where the "part" objects are an essential and inseparable component of the "whole" object.

- **Dependency:** It is a strong or dependent relationship.
- **Lifecycle:** The "part" object cannot exist meaningfully without the "whole" object. If the "whole" is destroyed, the "part" objects are also destroyed or cease to exist.
- **UML:** Represented by a solid diamond at the "whole" side of the association line.
- **Example:** A Car is composed of an Engine. If the Car object is destroyed, the Engine object is considered destroyed as well (it cannot function independently as a part of that car).

Difference between Association and Aggregation

#	Association	Aggregation
1	It is a general relationship between two independent classes.	It is a special form of association (a subtype).

#	Association	Aggregation
2	Represents a “uses-a” relationship.	Represents a “has-a” relationship.
3	Objects can be related, but the relationship is not necessarily whole-part.	Objects have a whole-part relationship (container-contained).
4	No ownership between objects.	Weak ownership – the container object, but not strongly.
5	Both objects are independent.	Both objects are also independent.
6	Example: A Teacher uses a Marker. They are just associated.	Example: A Department has Teachers (teachers still exist if department is deleted).

Difference between Aggregation and Generalization

#	Aggregation	Generalization
1	It is a whole-part (has-a) relationship.	It is an inheritance (is-a) relationship.
2	Represents that one class contains or owns another class.	Represents that one class inherits features of another class.
3	Both objects can exist independently.	Child class depends on the parent class for common structure/behaviour.
4	No concept of hierarchy.	Always forms a class hierarchy (superclass -> subclass).
5	Used to model object composition.	Used to model shared characteristics among classes.
6	Example: Department has Teachers -> Whole-part.	Example: Car is a Vehicle -> Inheritance.

What is an object? Why is Java called an object-oriented language?

=> An object is a real-world entity or instance of a class in Object-Oriented Programming. It contains:

- **State** (data/attributes) – Example: name, age, color.
- **Behaviour** (methods/functions) – Example: run(), eat(), display().
- **Identity** (unique address in memory) – Example: Every object is stored in a separate memory location.

Java is called an Object-Oriented Programming (OOP) language because:

- **Everything in Java is built around classes** – You cannot write any code without a class.
 - **Java supports all major OOP principles** – Encapsulation, Abstraction, Inheritance, Polymorphism.
 - **Real-world problems are solved using objects** – Java programs are organized around objects, not functions.
 - **Reusability through classes and inheritance** – Classes can be reused, extended, and modified.
 - **Java encourages modular and maintainable code** – Objects break the program into small, manageable parts.
-

Difference between final and finally

#	final	finally
1	A keyword used to apply restrictions on variables, methods, and classes.	A block used in exception handling to execute important code.

#	final	finally
2	Makes variables constant, methods non-overridable, and classes non-inheritable.	Always executes whether an exception occurs or not.
3	Used for immutability and security.	Used for cleanup code such as closing files, database connections, etc.
4	Part of OOP features.	Part of exception handling (try-catch-finally).
5	Related to class design.	Related to runtime execution flow.

Difference between throw and throws

#	throw	throws
1	Used to explicitly throw an exception from a method or block.	Used in method declaration to indicate that a method may throw exceptions.
2	Throws only one exception at a time.	Can declare multiple exceptions separated by commas.
3	Occurs inside a method or block of code.	Occurs in the method signature.
4	Used to throw custom exceptions.	Used to delegate exception handling to the caller.
5	Causes immediate interruption of the method.	Does not throw an exception by itself; only declares it.

Difference between Early and Late Binding

#	Early Binding	Late Binding
1	Also called Static Binding or Compile-time Binding.	Also called Dynamic Binding or Run-time Binding.
2	Method call is resolved at compile time.	Method call is resolved at run time.
3	Faster execution because decisions are made earlier.	Slower due to runtime decision-making.
4	Occurs in method overloading, constructors, and static methods.	Occurs in method overriding.
5	Less flexible but more efficient.	More flexible but slightly less efficient.

Difference between Abstract Class and Interface

#	Abstract Class	Interface
1	Supports both variables and methods with different access modifiers.	Variables are always public, static, and final (constants).
2	A class can extend only one abstract class (single inheritance).	A class can implement multiple interfaces (multiple inheritance).
3	Can have constructors.	Cannot have constructors.
4	Can maintain state using non-final variables.	Cannot maintain state; all variables are constants.
5	Performance is slightly faster than interfaces.	Performance may be slower due to dynamic resolution.

Difference between static and final keyword

#	Static	Final
1	Belongs to the class, not to individual objects.	Value/reference cannot be changed once assigned.
2	Shared by all objects of the class.	Provides constant or non-overridable behaviour.
3	Memory is allocated only once when the class is loaded.	Memory behaviour depends on how and where it is used.
4	A static variable can be modified.	A final variable cannot be modified.
5	A static class (nested) belongs to outer class without object creation.	A final class cannot be inherited.

What are exceptions? Explain the user-defined exceptions and system-defined exceptions with suitable examples.

=> Exceptions:

An exception is an unexpected or abnormal event that occurs during program execution and disrupts the normal flow of the program.

- Exceptions occur due to issues such as wrong input, file not found, division by zero etc.
- Java handles exceptions using the try-catch-finally mechanism.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException

Why Exceptions?

- To prevent abrupt termination of the program.
- To handle runtime errors smoothly.
- To provide meaningful error messages.

Types of Exceptions:

- System-defined (built-in) exceptions
- User-defined (custom) exceptions

1. System-Defined (Built-in) Exceptions

These are exceptions provided by Java's standard library. They occur automatically when a runtime error happens.

Example:

```
public class SystemExceptionExample {  
    public static void main(String[] args) {  
        int a = 10/0;  
        System.out.println("This will not execute");  
    }  
}
```

2. User-Defined (Custom) Exceptions

When built-in exceptions are not enough, programmers create their own exceptions.

To create a custom exception:

- Create a class extending Exception.

- Throw the exception using the throw keyword.
- Handle it using try-catch.

Example:

```
class AgeException extends Exception {
    AgeException(String msg) {
        super(msg);
    }
}

public class Test {
    static void checkAge(int age) throws AgeException {
        if (age < 18)
            throw new AgeException("Not eligible!");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Dynamic Binding

Dynamic Binding, also called Late Binding or Run-time Binding, is a process where the method to be executed is determined at run time, not at compile time.

In Java, dynamic binding occurs when a parent class reference refers to a child class object, and the overridden method in the child class is executed at runtime. It is used to achieve runtime polymorphism.

Key points:

- Happens during runtime.
- Works with method overriding.
- Decision is based on the actual object, not reference type.
- Provides flexibility and extensibility in programs.

Example:

```
class Parent {
    void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    void show() { System.out.println("Child"); }
}

Parent p = new Child();
p.show();
```